SecureChat - Assignment #2
Information Security (Fall 2025)
Technical Documentation & Software Manual

Name: *Tauha Imran*
Roll No: *22i-1239*
Section: *Your Roll No*
Repository: *https://github.com/tauhaimran/securechat-A2*

TABLE OF CONTENTS

1.Assignment Overview

SecureChat is a secure end-to-end communication system designed to demonstrate key Information Security principles including **confidentiality**, **authentication**, **integrity**, and **non-repudiation**.
The project implements a full client–server secure messaging workflow using modern cryptographic primitives such as AES, Diffie–Hellman, RSA-based PKI, digital signatures, and certificate validation.

This assignment focuses on building:

- A **secure handshake protocol** (PKI + DH key exchange)
- An **encrypted messaging channel** (AES-128-ECB with PKCS#7 padding)
- A **certificate-based trust system** using a custom Root CA
- **Secure storage and transcript logging**
- Complete **testing** of all security components

The end result is a working secure chat application that enforces strong cryptographic guarantees and showcases security best-practices.

1.1 System Architecture

SecureChat follows a modular, layered architecture where each security component is isolated and testable.
The overall workflow is:

1. Client Initiation

- Client loads its certificate and private key
- Sends a **Hello** message containing its certificate
- The server validates it using the Root CA

2. Server Authentication

- Server returns its own certificate (ServerHello)
- Client validates the server identity and certificate chain

3. Diffie–Hellman Key Exchange

- Client sends DH parameters (p, g, A)
- Server responds with its DH public value (B)
- Both derive a shared AES-128 key using SHA-256 → 16 bytes

4. Encrypted Communication

- All messages are encrypted with AES-128-ECB + PKCS#7
- Chat messages use a structured JSON protocol
- Each message includes a digital signature for integrity and non-repudiation

5. Transcript Logging & Receipts

- Every encrypted message and receipt is stored in a transcript object

- Receipts include:
    - user identity
    - transcript hash
    - RSA signature

This allows both parties to later prove message delivery and content authenticity.

6. Storage Layer

- Simple database wrapper for storing:
    - Users
    - Password hashes
    - Message transcripts

This architecture ensures clean separation of:

Layer	Description
Crypto Layer	AES, DH, Signatures, PKI
Protocol Layer	Pydantic models, message formats
Network Layer	Client/server socket communication
Storage Layer	Transcript + database handling

Scripts	Certificate creation and debugging

1.2 Features Implemented

Security & Cryptography

- AES-128-ECB encryption with PKCS#7 padding
- Secure Diffie–Hellman key exchange (2048-bit MODP Prime)
- RSA-based PKI with custom Root CA
- Certificate verification:
    - signature validation
    - CA trust validation
    - hostname validation (CN / SAN)
    - validity period checks
- RSA digital signatures (SHA-256, PKCS#1 v1.5)

Messaging Protocol

- Authenticated handshake using certificates
- Encrypted and signed messages
- Delivery receipts with transcript hashing
- Pydantic-based structured protocol models

Server Functionality

- User registration and login (with hashed passwords)
- Secure session establishment
- Transcript logging per session
- User identity verification via certificate matching

Client Functionality

- Establish secure connections
- Validate server certificate
- Derive symmetric encryption keys
- Send encrypted signed messages
- Display receipts and transcript verification

Scripts & Utilities

- `gen_ca.py` — create self-signed Root CA
- `gen_cert.py` — issue server/client certificates
- Diagnostic script for PKI debugging
- Base64 helpers, hashing utilities, timestamping

Testing

- Encryption/decryption tests
- DH shared secret agreement tests
- PKI and certificate-validation tests

- Message signature tests
- Full client–server integration tests
- Database and transcript verification tests

Together, these features satisfy the core security principles:

Principle	Provided By
Confidentiality	AES encryption
Integrity	SHA-256 + signatures
Authentication	PKI certificates
Non-Repudiation	Digital signatures + transcripts

Quick Summary of Directories

- **app/** — main application code
- **app/crypto/** — all cryptography implementations
- **app/common/** — protocol + utilities
- **app/storage/** — message/database storage
- **scripts/** — certificate generation, CA creation
- **certs/** — Root CA + server/client certificates
- **transcripts/** — saved chat transcripts
- **tests/** — functional and security tests

Repository Structure

```
securechat-A2/
├─ README.md
├─ requirements.txt
├─ .gitignore
├─ certs/
│   ├─ MyRootCA_ca_cert.pem
│   ├─ MyRootCA_ca_key.pem
│   ├─ client.example.com_cert.pem
│   ├─ client.example.com_key.pem
│   ├─ myserver.example.com_cert.pem
│   ├─ myserver.example.com_key.pem
├─ transcripts/
│   └─ demo_session.json
├─ scripts/
│   ├─ gen_ca.py
│   ├─ gen_cert.py
│   ├─ diag_pki.py
│   ├─ cli_commands.txt
├─ scripts_ss/                  # Support / dev scripts
├─ app/
│   ├─ .env
│   ├─ client.py
│   ├─ server.py
│   ├─ app_ss/                  # Reference / assignments
│   ├─ common/
│   ├─ protocol.py
│   ├─ utils.py
│   ├─ common_ss/
│   ├─ crypto/
│   │   ├─ aes.py
│   │   ├─ dh.py
│   │   ├─ pki.py
│   │   ├─ sign.py
│   ├─ crypto_ss/
│   ├─ storage/
│   │   ├─ db.py
│   │   ├─ transcript.py
│   ├─ testing_files/
├─ tests/
│   └─ manual/
└─ NOTES.md
```

2.Software Implementation Tables

2.1. app/client.py

— add in

2.2. app/server.py

— add in

2.3 app/crypto

2.3.1 crypto/aes.py

AES-128-ECB Helpers with PKCS#7:

- `pkcs7_pad(data: bytes) -> bytes` **— Pad input bytes to 16-byte block with PKCS#7**
- `pkcs7_unpad(data: bytes) -> bytes` **— Remove PKCS#7 padding; raises** `ValueError` **if invalid**
- `encrypt_aes(key: bytes, plaintext: bytes) -> bytes` **— AES-128-ECB encrypt with PKCS#7 padding**
- `decrypt_aes(key: bytes, ciphertext: bytes) -> bytes` **— AES-128-ECB decrypt and remove PKCS#7 padding**

Notes:

- `key` **must be exactly 16 bytes (AES-128).**
- `encrypt_aes` **outputs raw bytes (can be base64-encoded for JSON transport).**
- `decrypt_aes` **returns the original plaintext bytes.**

2.3.2 crypto/dh.py

Constants:

- `P_2048` **— 2048-bit MODP safe prime**
- `G = 2` **— generator**

Functions:

- `generate_dh_pair() -> Tuple[dh.DHPrivateKey, dh.DHPublicKey]` **— Generate fresh DH key pair**
- `get_dh_public_bytes(public_key: dh.DHPublicKey) -> int` **— Extract public value as integer**
- `compute_shared_secret(own_private_key: dh.DHPrivateKey, peer_public_value: int) -> bytes` **— Compute raw shared secret**
- `derive_aes_key(shared_secret: bytes) -> bytes` **— Truncate SHA-256 to 16 bytes for AES**
- `client_dh_initiate() -> Tuple[dh.DHPrivateKey, dict]` **— Client DH keypair + first message**
- `server_dh_respond(client_msg: dict, server_private_key: dh.DHPrivateKey) -> Tuple[bytes, dict]` **— Server DH response + AES key**
- `client_dh_finalize(client_private_key: dh.DHPrivateKey, server_msg: dict) -> bytes` **— Client computes AES key from server reply**

2.3.3 crypto/pki.py

Functions

- `load_certificate(pem_path: str) -> Certificate` **— Load X.509 certificate from PEM**

- `load_ca_certificate(ca_pem_path: str) -> Certificate` — Load trusted Root CA cert
- `verify_signature(ca_cert: Certificate, leaf_cert: Certificate) -> bool` — Check leaf signed by CA
- `check_validity(cert: Certificate) -> bool` — Check cert validity window
- `get_common_name(cert: Certificate) -> Optional[str]` — Extract CN from subject
- `get_san_dns_names(cert: Certificate) -> List[str]` — Get DNS names from SAN
- `verify_identity(cert: Certificate, expected_hostname: str) -> bool` — Check CN/SAN matches hostname
- `is_ca_certificate(cert: Certificate) -> bool` — Check BasicConstraints ca=True
- `validate_certificate(leaf_pem_path: str, ca_pem_path: str, expected_hostname: str) -> Certificate` — Full cert validation (chain, CA, hostname)
- `validate_server_certificate(server_cert_pem: str, ca_pem_path: str, expected_server_name: str) -> Certificate` — Client-side server cert check
- `validate_client_certificate(client_cert_pem: str, ca_pem_path: str, expected_client_name: Optional[str] = None) -> Certificate` — Server-side client cert check

2.3.4 crypto/sign.py

Functions

- `sign_data(private_key_pem: str, data: bytes) -> bytes` — Sign data using RSA PKCS#1 v1.5 SHA-256

- `verify_signature(public_key_pem: str, data: bytes, signature: bytes) -> bool` — **Verify RSA PKCS#1 v1.5 SHA-256 signature**

2.4 app/common

2.4.1 common/protocol.py

Pydantic Models

- `BaseMessage(type: str, timestamp: int)` — **Base message class for all messages**
- `Hello(client_cert: str)` — **Client hello / handshake message**
- `ServerHello(server_cert: str)` — **Server hello / handshake message**
- `Register(username: str, password_hash: str)` — **User registration message**
- `Login(username: str, password_hash: str)` — **User login message**
- `DHClient(p: str, g: str, A: str)` — **Client DH key exchange message**
- `DHServer(B: str)` — **Server DH key exchange message**
- `Message(sender: str, ciphertext: str)` — **Encrypted chat message**
- `Receipt(identity: str, transcript_hash: str, signature: str)` — **Message receipt / acknowledgement**

2.4.2 common/utils.py

Helper Functions:

- `now_ms() -> int` — **Current time in milliseconds**

- `b64e(b: bytes) -> str` — **Base64 encode bytes to string**
- `b64d(s: str) -> bytes` — **Base64 decode string to bytes**
- `sha256_hex(data: bytes) -> str` — **SHA-256 hash as hex string**

2.5. scripts/

2.5.1 scripts/gen_ca.py

Root CA Creation:

- `create_root_ca(ca_name: str, output_dir: str = "certs") -> None` — **Generate self-signed RSA Root CA**

2.5.2 scripts/gen_cert.py

Issue Server/Client Certificate Signed by Root CA:

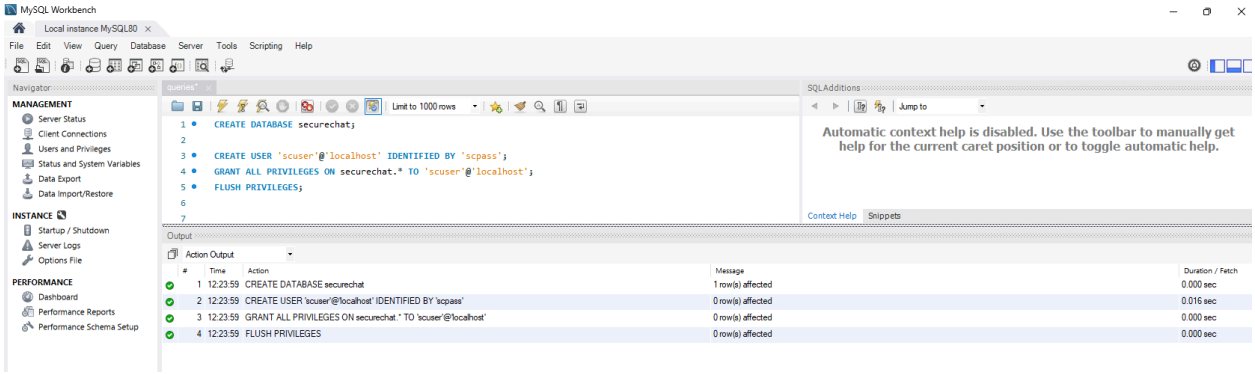
- `issue_certificate(ca_name: str, cert_name: str, cert_type: str, output_dir: str = "certs") -> None` — **Generate server/client X.509 certificate**

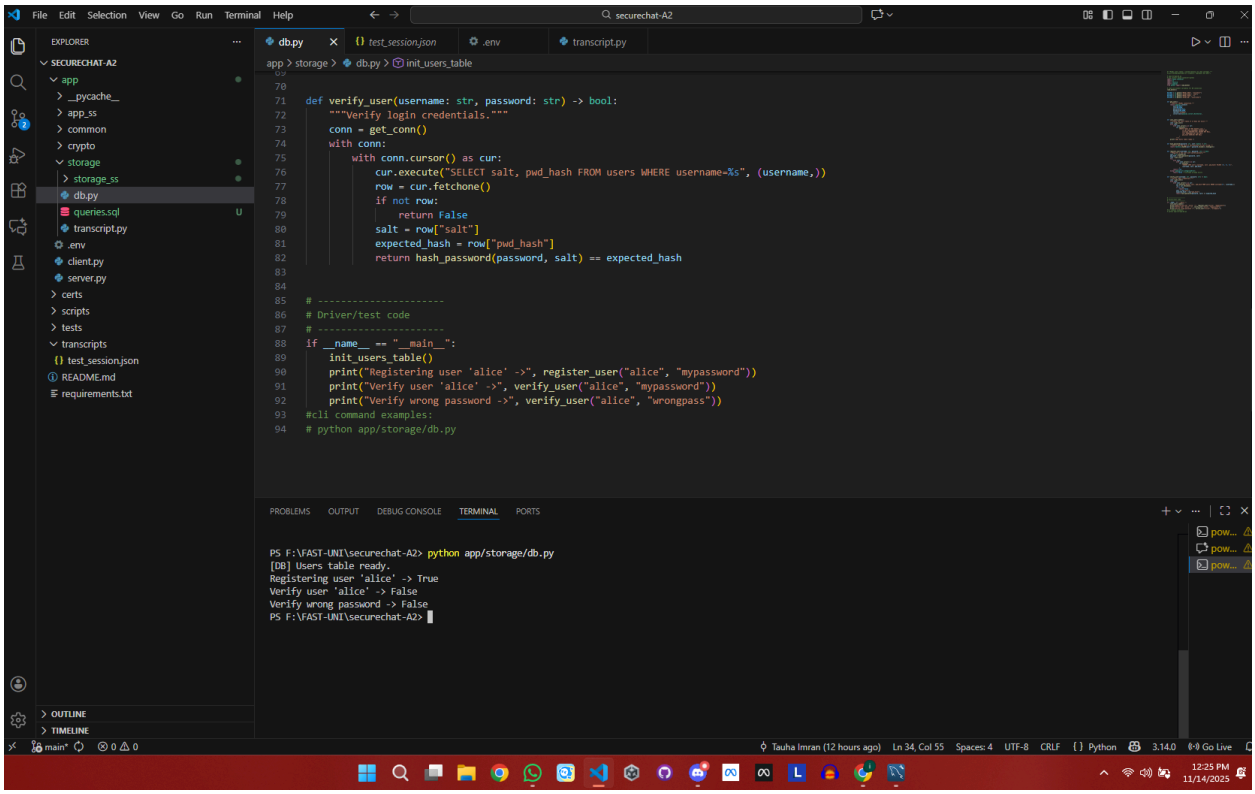
---

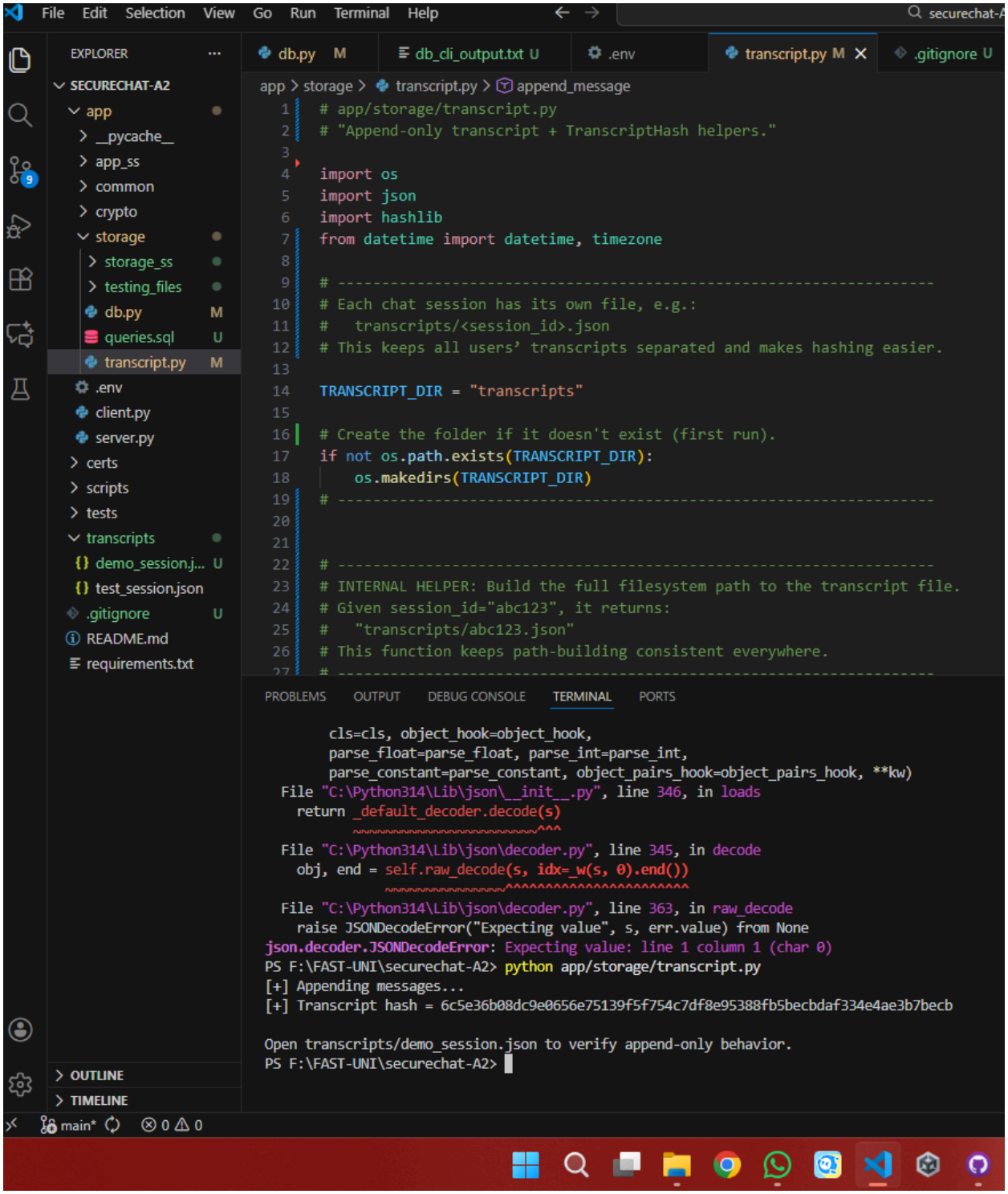
3. Evidence

The following screenshots are tests of individual segments and collective ones too covering all aspects of of confidentiality, Authenticity , Integrity and Non-Repudation

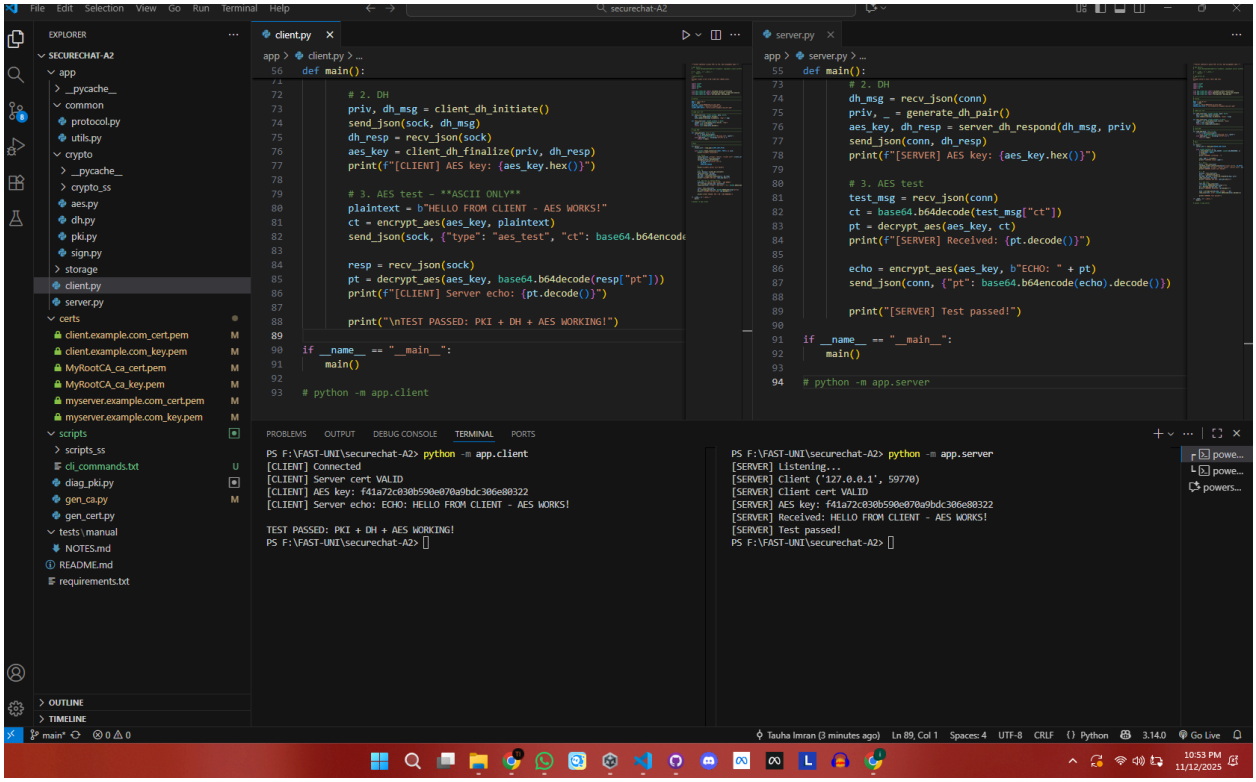
3.1 Storage and Database tests


MySQL Workbench
Local instance MySQL80
File Edit View Query Database Server Tools Scripting Help
Navigator
MANAGEMENT
Server Status
Client Connections
Users and Privileges
Status and System Variables
Data Export
Data Import/Restore
INSTANCE
Startup / Shutdown
Server Logs
Options File
PERFORMANCE
Dashboard
Performance Reports
Performance Schema Setup
queries*
Limit to 1000 rows
1 CREATE DATABASE securechat;
2
3 CREATE USER 'scuser'@'localhost' IDENTIFIED BY 'scpass';
4 GRANT ALL PRIVILEGES ON securechat.* TO 'scuser'@'localhost';
5 FLUSH PRIVILEGES;
6
7
SQLAdditions
Jump to
Automatic context help is disabled. Use the toolbar to manually get help for the current caret position or to toggle automatic help.
Context Help
Snippets
Output
Action Output
Time Action Message Duration / Fetch
1 12:23:59 CREATE DATABASE securechat 1 row(s) affected 0.000 sec
2 12:23:59 CREATE USER 'scuser'@'localhost' IDENTIFIED BY 'scpass' 0 row(s) affected 0.016 sec
3 12:23:59 GRANT ALL PRIVILEGES ON securechat.* TO 'scuser'@'localhost' 0 row(s) affected 0.000 sec
4 12:23:59 FLUSH PRIVILEGES 0 row(s) affected 0.000 sec


File Edit Selection View Go Run Terminal Help
securechat-A2
EXPLORER
SECURECHAT-A2
app
__pycache__
app_ss
common
crypto
storage
storage_ss
db.py
queries.sql
transcript.py
.env
client.py
server.py
certs
scripts
tests
transcripts
test_session.json
README.md
requirements.txt
db.py
test_session.json
.env
transcript.py
app > storage > db.py > init_users_table
def verify_user(username: str, password: str) -> bool:
"""Verify login credentials."""
conn = get_conn()
with conn:
with conn.cursor() as cur:
cur.execute("SELECT salt, pwd_hash FROM users WHERE username=%s", (username,))
row = cur.fetchone()
if not row:
return False
salt = row["salt"]
expected_hash = row["pwd_hash"]
return hash_password(password, salt) == expected_hash
# ----------------------
# Driver/test code
# ----------------------
if __name__ == "__main__":
init_users_table()
print("Registering user 'alice' ->", register_user("alice", "mypassword"))
print("Verify user 'alice' ->", verify_user("alice", "mypassword"))
print("Verify wrong password ->", verify_user("alice", "wrongpass"))
#cli command examples:
python app/storage/db.py
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS
PS F:\FAST-UNI\securechat-A2> python app/storage/db.py
[DB] Users table ready.
Registering user 'alice' -> True
Verify user 'alice' -> False
Verify wrong password -> False
PS F:\FAST-UNI\securechat-A2>
OUTLINE
TIMELINE
main*
Tauha Imran (12 hours ago)
Ln 34, Col 55
Spaces: 4
UTF-8
CRLF
Python
3.14.0
Go Live
12:25 PM
11/14/2025



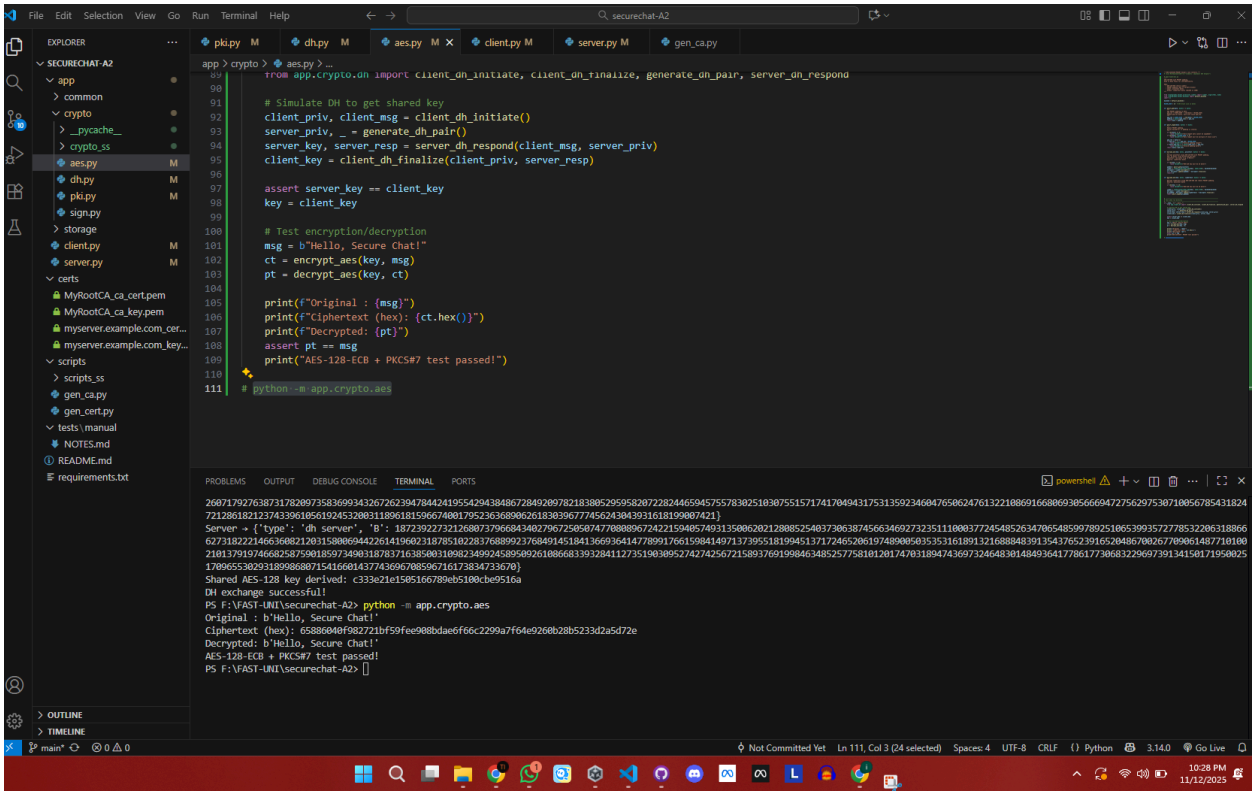
3.2 Full application Tests

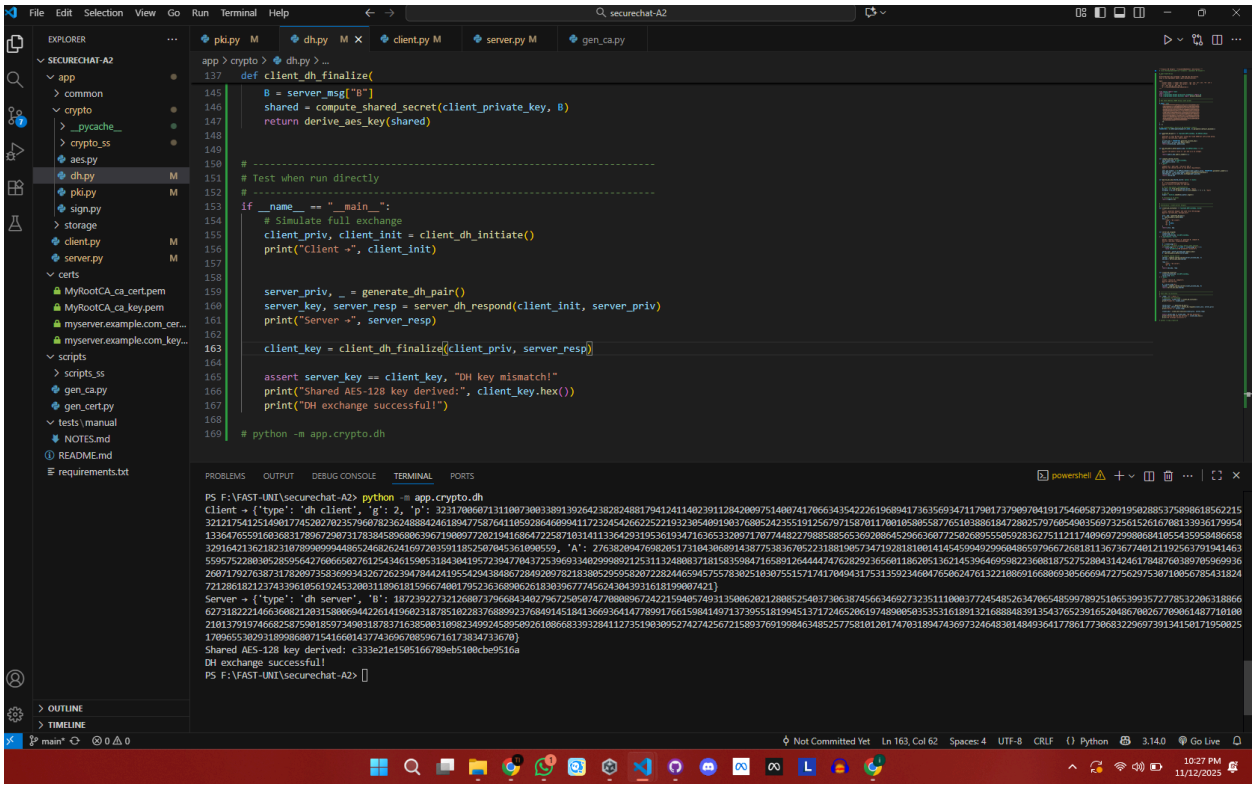


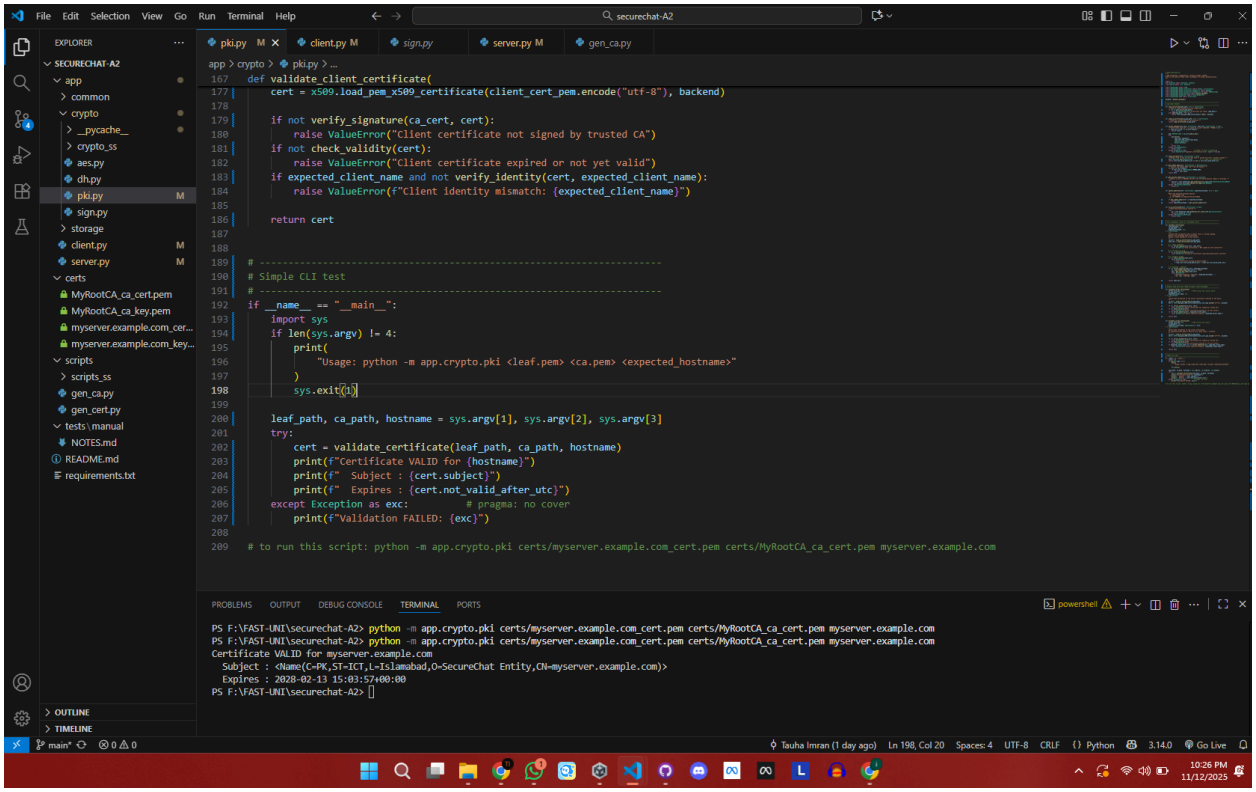

```
PS F:\FAST-UNI\securechat-A2> python -m app.client
[CLIENT] Connected
[CLIENT] Server cert OK

============================
WELCOME TO SECURE CHAT APP
============================
Press L for Login or R for Register: R
Username: tauha
Password:
[SERVER] SUCCESS
You can now login.

============================
WELCOME TO SECURE CHAT APP
============================
Press L for Login or R for Register: L
Username: tauha
Password:
[SERVER] Login success!
[CLIENT] Sent DH init
[CLIENT] AES key ready

Chat started! Type 'I want to exit' to quit.
[YOU] I hate info sex
[SERVER] me too mf
[YOU] fuck it
[SERVER] SUBMIT IT
[YOU] sybau
[SERVER] no
[YOU] exit
[SERVER] exit
[YOU] I want to exit
[SERVER] Session closed. Transcript hash: 702948e7dfc6a930b3b0d9669eabbf0291e
f558c5a1dd08a271622ea9441ea97
PS F:\FAST-UNI\securechat-A2>
```

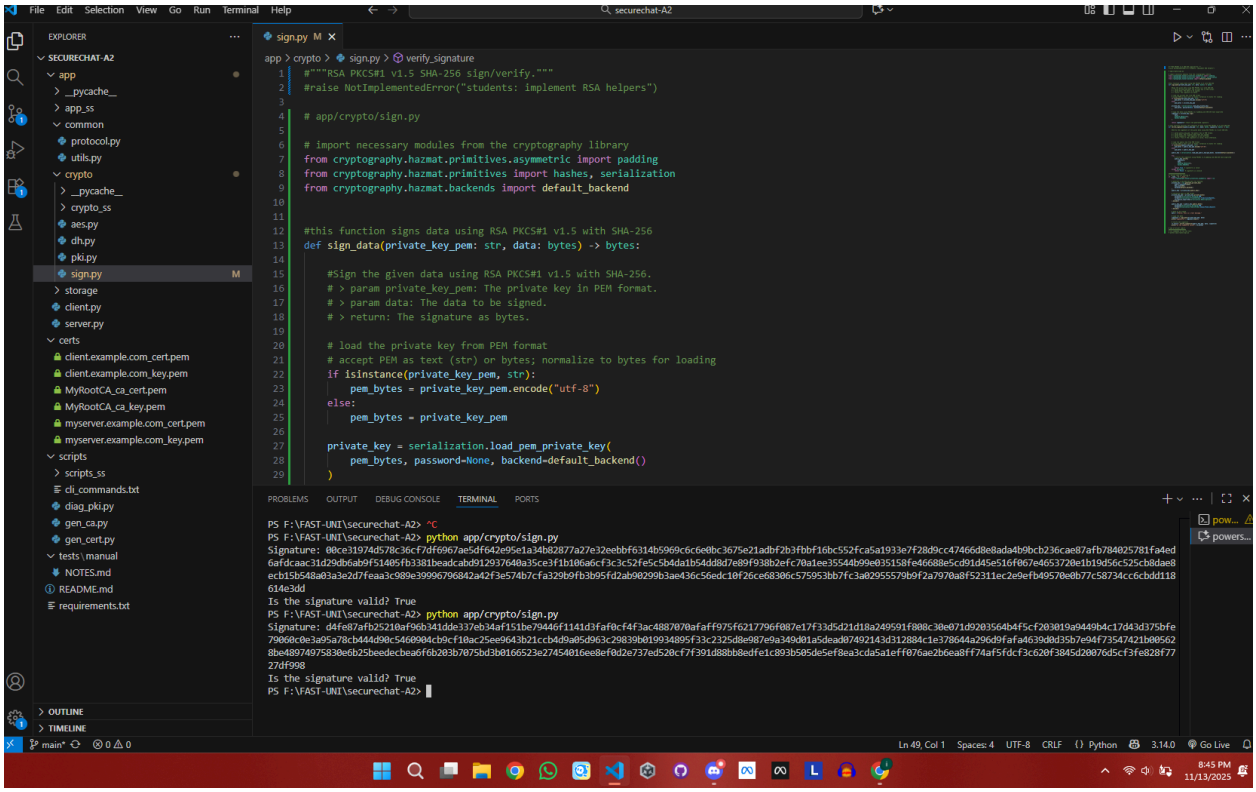
```
PS F:\FAST-UNI\securechat-A2> python -m app.server
[DB] Users table ready.
[SERVER] Listening on 127.0.0.1:9999
[SERVER] New client: ('127.0.0.1', 59721)
[SERVER] Client ('127.0.0.1', 59721) cert OK
[SERVER] Sent server hello

--- REGISTER DEBUG ---
salt (hex)      = b8c9f7c96809a2306219327c456898f1
pwd_hash        = bc10e7aed148917dcf7094ae3ba1ab1b12245cb0fd67b4a06b8042e6bca
62632
----------------------

[SERVER] Registered tauha

--- LOGIN DEBUG ---
salt from DB (raw) = b'\xb8\xc9\xf7\xc9h\t\xa20b\x192|Eh\x98\xf1' <class 'byt
es'>
salt converted     = b8c9f7c96809a2306219327c456898f1
expected hash      = bc10e7aed148917dcf7094ae3ba1ab1b12245cb0fd67b4a06b8042e6
bca62632
computed hash      = bc10e7aed148917dcf7094ae3ba1ab1b12245cb0fd67b4a06b8042e6
bca62632
--------------------

[SERVER] Login OK: tauha
[SERVER] DH completed
[SERVER] Chat with ('127.0.0.1', 59721). Type messages. 'exit' to end.
[CLIENT] I hate info sex
[SERVER] me too mf
[CLIENT] fuck it
[SERVER] SUBMIT IT
[CLIENT] sybau
[SERVER] no
[CLIENT] exit
[SERVER] exit
[CLIENT] I want to exit
[SERVER] Sent receipt: 702948e7dfc6a930b3b0d9669eabbf0291ef558c5a1dd08a271622
ea9441ea97
[SERVER] Closed ('127.0.0.1', 59721)
```

3.3 Cryptography Tests

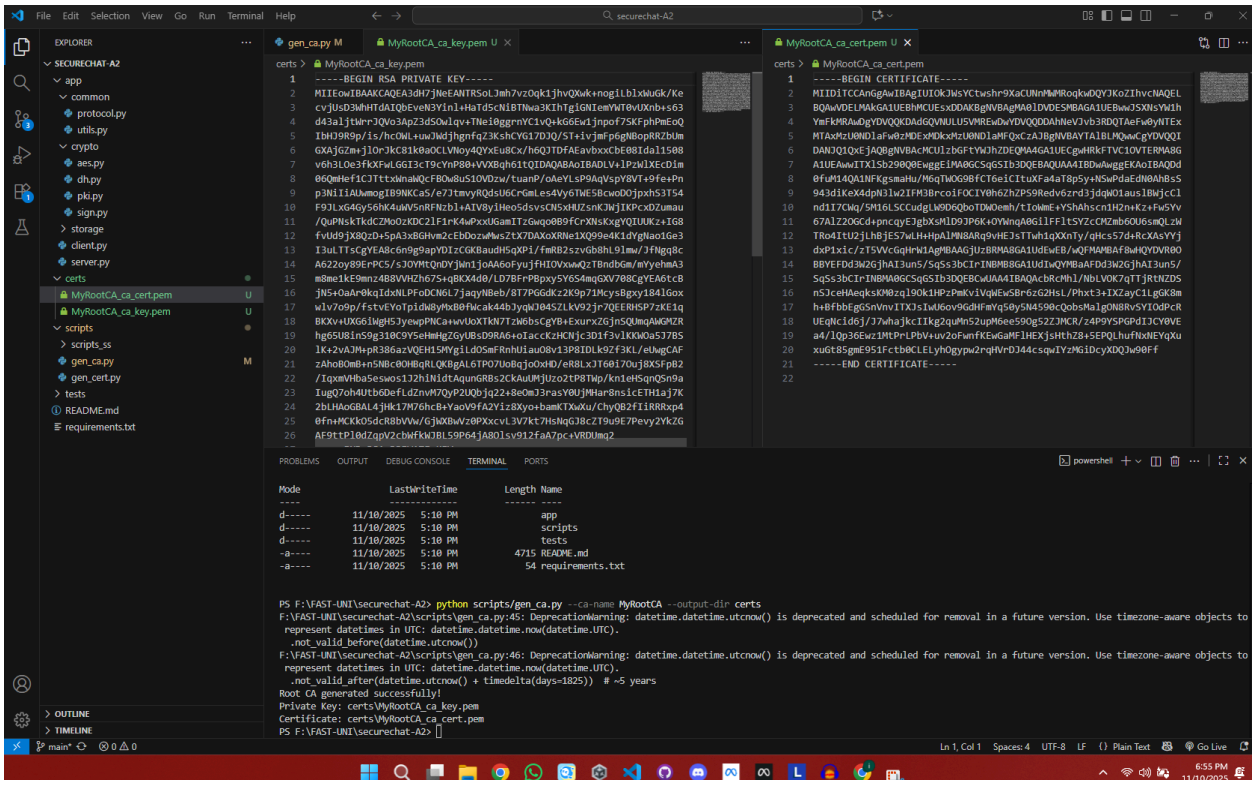





File Edit Selection View Go Run Terminal Help
securechat-A2
EXPLORER
SECURECHAT-A2
app
common
crypto
__pycache__
crypto_ss
aes.py
dh.py
pki.py
sign.py
storage
client.py
server.py
certs
MyRootCA_ca_cert.pem
MyRootCA_ca_key.pem
myserver.example.com_cer...
myserver.example.com_key...
scripts
scripts_ss
gen_ca.py
gen_cert.py
tests \ manual
NOTES.md
README.md
requirements.txt
pki.py M
client.py M
sign.py
server.py M
gen_ca.py
app > crypto > pki.py > ...
def validate_client_certificate(
cert = x509.load_pem_x509_certificate(client_cert_pem.encode("utf-8"), backend)
if not verify_signature(ca_cert, cert):
raise ValueError("Client certificate not signed by trusted CA")
if not check_validity(cert):
raise ValueError("Client certificate expired or not yet valid")
if expected_client_name and not verify_identity(cert, expected_client_name):
raise ValueError(f"Client identity mismatch: {expected_client_name}")
return cert
# --
# Simple CLI test
# --
if __name__ == "__main__":
import sys
if len(sys.argv) != 4:
print(
"Usage: python -m app.crypto.pki <leaf.pem> <ca.pem> <expected_hostname>"
)
sys.exit(1)
leaf_path, ca_path, hostname = sys.argv[1], sys.argv[2], sys.argv[3]
try:
cert = validate_certificate(leaf_path, ca_path, hostname)
print(f"Certificate VALID for {hostname}")
print(f" Subject : {cert.subject}")
print(f" Expires : {cert.not_valid_after_utc}")
except Exception as exc: # pragma: no cover
print(f"Validation FAILED: {exc}")
# to run this script: python -m app.crypto.pki certs/myserver.example.com_cert.pem certs/MyRootCA_ca_cert.pem myserver.example.com
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS
powershell
PS F:\FAST-UNI\securechat-A2> python -m app.crypto.pki certs/myserver.example.com_cert.pem certs/MyRootCA_ca_cert.pem myserver.example.com
PS F:\FAST-UNI\securechat-A2> python -m app.crypto.pki certs/myserver.example.com_cert.pem certs/MyRootCA_ca_cert.pem myserver.example.com
Certificate VALID for myserver.example.com
Subject : <Name(C=PK,ST=ICT,L=Islamabad,O=SecureChat Entity,CN=myserver.example.com)>
Expires : 2028-02-13 15:03:57+00:00
PS F:\FAST-UNI\securechat-A2>
OUTLINE
TIMELINE
main*
Tauha Imran (1 day ago)
Ln 198, Col 20
Spaces: 4
UTF-8
CRLF
Python
3.14.0
Go Live
10:26 PM
11/12/2025


File Edit Selection View Go Run Terminal Help
securechat-A2
EXPLORER
SECURECHAT-A2
app
__pycache__
app_ss
common
protocol.py
utils.py
crypto
__pycache__
crypto_ss
aes.py
dh.py
pki.py
sign.py
storage
client.py
server.py
certs
client.example.com_cert.pem
client.example.com_key.pem
MyRootCA_ca_cert.pem
MyRootCA_ca_key.pem
myserver.example.com_cert.pem
myserver.example.com_key.pem
scripts
scripts_ss
cli_commands.txt
diag_pki.py
gen_ca.py
gen_cert.py
tests \ manual
NOTES.md
README.md
requirements.txt
sign.py M
app > crypto > sign.py > verify_signature
#"""RSA PKCS#1 v1.5 SHA-256 sign/verify."""
#raise NotImplementedError("students: implement RSA helpers")
# app/crypto/sign.py
# import necessary modules from the cryptography library
from cryptography.hazmat.primitives.asymmetric import padding
from cryptography.hazmat.primitives import hashes, serialization
from cryptography.hazmat.backends import default_backend
#this function signs data using RSA PKCS#1 v1.5 with SHA-256
def sign_data(private_key_pem: str, data: bytes) -> bytes:
#Sign the given data using RSA PKCS#1 v1.5 with SHA-256.
# > param private_key_pem: The private key in PEM format.
# > param data: The data to be signed.
# > return: The signature as bytes.
# load the private key from PEM format
# accept PEM as text (str) or bytes; normalize to bytes for loading
if isinstance(private_key_pem, str):
pem_bytes = private_key_pem.encode("utf-8")
else:
pem_bytes = private_key_pem
private_key = serialization.load_pem_private_key(
pem_bytes, password=None, backend=default_backend()
)
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS
PS F:\FAST-UNI\securechat-A2> ^C
PS F:\FAST-UNI\securechat-A2> python app/crypto/sign.py
Signature: 00ce31974d578c36cf7df6967ae5df642e95e1a34b82877a27e32eebbf6314b5969c6c6e0bc3675e21adbf2b3fbbf16bc552fca5a1933e7f28d9cc47466d8e8ada4b9bcb236cae87afb784025781fa4ed6afdcaac31d29d0f6ab9f51405fb3381bead6cab912937640a35ce3f1b106a6cf3c3c52fe5c5b4da1b54dd8d7e89f938b2efc70a1ee35544b99e035158fe46688e5cd01d45e516f067e4653720e1b19d56c525cbd6dae8ecb15b548a03a3e2d7feaa3c989e39996796842a42f3e574b7cfa329b9fb3b95fd2ab90299b3ae436c56edc10f26ce68306c575953bb7fc3a02955579b9f2a7970a8f52311ec2e9efb49570e0b77c58734cc6cbdd118614e3dd
Is the signature valid? True
PS F:\FAST-UNI\securechat-A2> python app/crypto/sign.py
Signature: d4fe87afb25210af96b341dde337eb34af151be79446f1141d3faf0cf4f3ac4887070afaff975f6217796f087e17f33d5d21d18a249591f808c30e071d9203564b4f5cf203019a9449b4c17d43d375bfe79060c0e3a95a78cb444d90c5460904cb9cf10ac25ee9643b21ccb4d9a05d963c29839b019934895f33c2325d8e987e9a349d01a5dead07492143d312884c1e378644a296d9fafa4639dd35b7e94f7354721b00562 8be48974975830e6b25beedecbea6f6b203b7075bd3b0166523e27454016ee8ef0d2e737ed520cf7f391d88bb8edfe1c893b505de5ef8ea3cda5a1eff076ae2b6ea8ff74af5fdcf3c620f3845d20076d5cf3fe828f7727df998
Is the signature valid? True
PS F:\FAST-UNI\securechat-A2>
OUTLINE
TIMELINE
main*
Ln 49, Col 1
Spaces: 4
UTF-8
CRLF
Python
3.14.0
Go Live
8:45 PM
11/13/2025

3.4 Scripts and Certificates tests



3.5 Common Functionality Tests